

```
In [1]: #Step 1: Import Required Libraries
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
```

```
In [2]: #Step 2: Load the Datasets
air_df = pd.read_csv("AirQualityUCI.csv")

print(air_df.head())
```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	Wind	PT08.S2(NMH
C) \							
0	10-03-2004	18:00:00	2.6	1360	150	11.9	104
6							
1	10-03-2004	19:00:00	2.0	1292	112	9.4	95
5							
2	10-03-2004	20:00:00	2.2	1402	88	9.0	93
9							
3	10-03-2004	21:00:00	2.2	1376	80	9.2	94
8							
4	10-03-2004	22:00:00	1.6	1272	51	6.5	83
6							

	Solor.R	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	Ozone	Temp	RH	AH
0	166	1056	113	1692	1268	13.6	48.9	0.7578
1	103	1174	92	1559	972	13.3	47.7	0.7255
2	131	1140	114	1555	1074	11.9	54.0	0.7502
3	172	1092	122	1584	1203	11.0	60.0	0.7867
4	131	1205	116	1490	1110	11.2	59.6	0.7888

```
In [12]: #Load Heart Disease Dataset
heart_df = pd.read_csv("heartdisease.csv")
heart_df = heart_df.replace('?', np.nan)
print(heart_df.head())
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	sl
ope \											
0	63	1	1	145	233	1	2	150	0	2.3	
3											
1	67	1	4	160	286	0	2	108	1	1.5	
2											
2	67	1	4	120	229	0	2	129	1	2.6	
2											
3	37	1	3	130	250	0	0	187	0	3.5	
3											
4	41	0	2	130	204	0	2	172	0	1.4	
1											

	ca	thal	num
0	0	6	0
1	3	3	2
2	2	7	1
3	0	3	0
4	0	3	0

In [13]: *# Step 3: Check Rows and Columns*

```
#Air Quality Dataset
print("Air Quality Dataset Shape:")
print(air_df.shape)

print("\nColumns:")
print(air_df.columns)

#Heart Disease Dataset
print("Heart Disease Dataset Shape:")
print(heart_df.shape)

print("\nColumns:")
print(heart_df.columns)
```

Air Quality Dataset Shape:
(9357, 16)

Columns:
Index(['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'Wind',
 'PT08.S2(NMHC)', 'Solor.R', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO
2)',
 'Ozone', 'Temp', 'RH', 'AH', 'id'],
 dtype='str')
Heart Disease Dataset Shape:
(303, 14)
Columns:
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalac
h',
 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'],
 dtype='str')

In [14]: *#a. Data Cleaning*

```
#Air Quality Data Cleaning:

#step1: Check missing values
print(air_df.isnull().sum())

#step2: Remove Missing Values
air_df = air_df.dropna()

print(air_df.shape)

#step3:Remove Duplicate Rows
air_df = air_df.drop_duplicates()

print(air_df.shape)
```

```

Date          0
Time          0
CO(GT)        0
PT08.S1(CO)   0
NMHC(GT)      0
Wind          0
PT08.S2(NMHC) 0
Solor.R       0
PT08.S3(NOx)  0
NO2(GT)       0
PT08.S4(NO2)  0
Ozone         0
Temp          0
RH            0
AH            0
id            0
dtype: int64
(9357, 16)
(9357, 16)

```

```

In [15]: #a. Data Cleaning

#Heart Disease Data Cleaning

print(heart_df.isnull().sum())

#step1: Remove Missing Values
heart_df = heart_df.dropna()

print(heart_df.shape)

```

```

age          0
sex          0
cp           0
trestbps     0
chol         0
fbs          0
restecg      0
thalach      0
exang        0
oldpeak      0
slope        0
ca           4
thal         2
num          0
dtype: int64
(297, 14)

```

```

In [16]: # b. Data Integration

#Add ID Columns
air_df['id'] = range(1, len(air_df)+1)

heart_df['id'] = range(1, len(heart_df)+1)

#Merge Datasets
merged_df = pd.merge(air_df, heart_df, on='id')

print(merged_df.head())

```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	Wind	PT08.S2(NMH
C) \							
0	10-03-2004	18:00:00	2.6	1360	150	11.9	104
6							
1	10-03-2004	19:00:00	2.0	1292	112	9.4	95
5							
2	10-03-2004	20:00:00	2.2	1402	88	9.0	93
9							
3	10-03-2004	21:00:00	2.2	1376	80	9.2	94
8							
4	10-03-2004	22:00:00	1.6	1272	51	6.5	83
6							

	Solor.R	PT08.S3(NOx)	NO2(GT)	...	chol	fbs	restecg	thalach	exang
\									
0	166	1056	113	...	233	1	2	150	0
1	103	1174	92	...	286	0	2	108	1
2	131	1140	114	...	229	0	2	129	1
3	172	1092	122	...	250	0	0	187	0
4	131	1205	116	...	204	0	2	172	0

	oldpeak	slope	ca	thal	num
0	2.3	3	0	6	0
1	1.5	2	3	3	2
2	2.6	2	2	7	1
3	3.5	3	0	3	0
4	1.4	1	0	3	0

[5 rows x 30 columns]

```
In [17]: # c. Data Transformation

# Convert Column Names to Lowercase
merged_df.columns = merged_df.columns.str.lower()

print(merged_df.columns)

#Normalize Numerical Data
scaler = StandardScaler()

heart_df[['age', 'chol', 'trestbps']] = scaler.fit_transform(
    heart_df[['age', 'chol', 'trestbps']]
)

print(heart_df.head())
```

```
Index(['date', 'time', 'co(gt)', 'pt08.s1(co)', 'nmhc(gt)', 'wind',
      'pt08.s2(nmhc)', 'solor.r', 'pt08.s3(nox)', 'no2(gt)', 'pt08.s4(no
2)',
      'ozone', 'temp', 'rh', 'ah', 'id', 'age', 'sex', 'cp', 'trestbps',
      'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope',
      'ca',
      'thal', 'num'],
      dtype='str')
      age  sex  cp  trestbps      chol  fbs  restecg  thalach  exang  \
0  0.936181    1   1  0.750380 -0.276443    1         2      150      0
1  1.378929    1   4  1.596266  0.744555    0         2      108      1
2  1.378929    1   4 -0.659431 -0.353500    0         2      129      1
3 -1.941680    1   3 -0.095506  0.051047    0         0      187      0
4 -1.498933    0   2 -0.095506 -0.835103    0         2      172      0

      oldpeak  slope  ca  thal  num  id
0         2.3      3   0     6    0   1
1         1.5      2   3     3    2   2
2         2.6      2   2     7    1   3
3         3.5      3   0     3    0   4
4         1.4      1   0     3    0   5
```

In [18]: *# d. Error Correcting*

#Check Negative Values

```
print((heart_df['chol'] < 0).sum())
```

#Replace Incorrect Negative Values

```
heart_df['chol'] = heart_df['chol'].apply(lambda x: abs(x))
```

```
print(heart_df['chol'].head())
```

#Check Data Types

```
print(heart_df.dtypes)
```

```
162
0      0.276443
1      0.744555
2      0.353500
3      0.051047
4      0.835103
Name: chol, dtype: float64
age      float64
sex      int64
cp      int64
trestbps float64
chol     float64
fbs      int64
restecg  int64
thalach  int64
exang    int64
oldpeak  float64
slope    int64
ca       str
thal     str
num      int64
id       int64
dtype: object
```

```
In [19]: # e. Data Model Building

#Define Features and Target
X = heart_df.drop('num', axis=1)

y = heart_df['num']

#Normalize Data
scaler = StandardScaler()

X = scaler.fit_transform(X)

#Split Training and Testing Data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

#Train Logistic Regression Model
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

#Predict Values
y_pred = model.predict(X_test)

#Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
```

Model Accuracy: 0.6333333333333333

In []: